Intent Engineering SKILL.md: Design Guide & Research Summary

Overview

The attached SKILL.md file is a complete, production-ready Claude Code skill (917 lines) that serves as an authoritative reference for intent engineering within your Claude Code setup. It synthesizes the leading frameworks—Huryn's 7-part structure, Pathmode's IntentSpec format, the I-5 Framework, and Action Schemas—into a unified 9-section template with validation checklists, domain examples, retrofitting guidance, and anti-pattern detection.

A companion blank template file is also included for quick-start use.

Source Framework Synthesis

The skill merges four distinct approaches into one coherent template. Huryn's framework provides the core 7-part agent intent structure: Objective, Desired Outcomes, Health Metrics, Strategic Context, Constraints (split into steering vs. hard), Decision Types & Autonomy (a four-level gradient from Full to Human-Required), and Stop Rules. Pathmode's IntentSpec adds User Goal (the job-to-be-done), Edge Cases as explicit boundary conditions, and Verification criteria—closing the gap between "what to achieve" and "how to confirm it's done". The I-5 Framework contributes the top-down flow from Intent → Interpretation → Information → Instruction → Implementation, which maps well to the skill's layered approach of defining purpose before constraints before execution boundaries. Action Schemas from GitHub's multi-agent engineering guide add typed, constrained output formats that eliminate ambiguous agent actions in structured workflows.[1][2][3][4][5][6]

The unified 9-section template covers: Objective, User Goal, Desired Outcomes, Health Metrics, Strategic Context, Constraints (Steering + Hard), Decision Authority, Edge Cases, and Stop Rules & Verification.

Key Design Decisions

Why 9 Sections Instead of 7

Huryn's original 7-part structure omits two things that consistently cause failures in practice. First, **User Goal** (from Pathmode) forces spec writers to articulate the job-to-be-done from the user's perspective, not the system's perspective—this is the difference between "categorize transactions" and "user can answer 'where did my money go?' in 30 seconds". Second, **Edge Cases** as a dedicated section ensures boundary conditions get explicit treatment rather than being scattered across constraints and stop rules. Every unhandled edge case is a potential hallucination point—the agent will encounter it eventually and invent behavior you didn't authorize.[2] [3]

Constraint Split: Steering vs. Hard

The most impactful principle in the skill is the distinction between steering constraints (prompt layer, influence reasoning) and hard constraints (architecture layer, enforce compliance). This addresses the #3 most common intent spec mistake: putting critical constraints in prompts where they're merely suggestions. The decision rule is simple: if violating a constraint would cause real harm (data loss, financial exposure, security breach), it must be enforced architecturally. Production agent failures consistently trace back to constraints that were technically stated but not mechanically enforced.[4][7][8][1]

Autonomy as a Gradient

The four-level autonomy model (Full → Guarded → Proposal-first → Human-required) is assigned using five risk lenses: blast radius, reversibility, confidence, precedent, and visibility. This replaces the binary "autonomous or not" approach that leaves dangerous gaps—particularly in the middle zone where actions are user-visible but not catastrophic. The security research strongly supports the inverse

correlation principle: as agent capability increases, autonomy must decrease proportionally to maintain safety.[9][10][4]

Quality Validation System

The skill includes a 30-item validation checklist organized across 7 dimensions (Objective, Outcome, Health Metric, Constraint, Autonomy, Stop Rule, and Edge Case quality). Each checklist item maps directly to a known failure mode documented in production agent analysis.[7][8]

Top 10 Mistakes

The skill codifies the ten most common intent spec failures, drawn from production analysis and practitioner experience:

Rank	Mistake	Root Cause
1	Outcomes are activities, not states	Confusing what the agent does with what exists after[1]
2	Missing health metrics	Goodhart vulnerability—agent games primary metric[1]
3	Hard constraints left in prompts	Critical rules enforced by "suggestion" not code[4]
4	Vague autonomy boundaries	"Use good judgment" is not a specification[9]
5	Stop rules only cover happy paths	No abort conditions for failures or edge cases[8]
6	Intent spec is really a detailed prompt	Steps masquerading as intent—no judgment framework[3]
7	Over-constraining	So many rules the agent can't exercise useful judgment[10]
8	Conflicting health metrics	Impossible optimization triangle[1]
9	Cargo cult intent engineering	Template structure without understanding why[11]
10	No verification criteria	Spec not specific enough to validate[2]

Domain Examples

The skill includes four complete worked examples spanning different agent domains:

- **Software Development Agent** (code review, PR management): Demonstrates autonomy graduation from auto-approving doc-only PRs to requiring human approval for auth/payments code, with health metrics preventing false-positive review fatigue.

- **Personal Productivity Agent** (daily planning, inbox processing): Shows read-only hard constraints preventing the agent from sending communications, with user override rate as a calibration health metric.
- **Creative Production Agent** (animation pipeline, asset management): Illustrates "never delete source files" as an architectural constraint and asset quality scores as health metrics against compression artifacts.
- **Financial Analysis Agent** (spending tracking, budget optimization): Demonstrates the "never guess on financial data" steering constraint and the critical distinction between categorizing (full autonomy) and providing investment advice (forbidden).

Retrofitting Process for Existing Skills

The skill provides a three-level conversion framework for your 106 existing SKILL.md files:[2]

Level	Effort	What to Add	Best For
Level 1: Minimum Viable	30 min/skill	Objective + Outcomes + Stop Rules (keep existing instructions below)	Simple, interactive skills
Level 2: Structured	2-4 hrs/skill	All of Level 1 + Health Metrics, Constraint split, Decision Authority, Edge Cases	Multi-step workflows
Level 3: Full Conversion	4-8 hrs/skill	Complete rewrite using 9-section template; original instructions dissolved	Autonomous, high-blast-radius skills

Prioritization order: (1) skills touching production/external APIs, (2) skills with frequent wrong outputs, (3) skills intended for autonomous operation, (4) complex multi-step workflows. Simple single-task skills used interactively only need Level 1.

The key test for a successful conversion: remove the original step-by-step instructions and see if the agent can still succeed using only the intent spec. If yes, the conversion is complete.

Anti-Pattern Detection

Five named anti-patterns are documented with detection criteria and fixes:

- **"Prompt in a Hat"**: Step-by-step instructions with intent section headers added. Fix: delete the steps, write the why, let the agent figure out the how.[3]
- **"Over-Constrained Straitjacket"**: So many constraints the agent can't make useful decisions. Fix: for each constraint, ask "if the agent did something different, would that be bad?"[10]
- **"Conflicting Health Metrics"**: Metrics that can't all be satisfied simultaneously. Fix: explicit priority ordering.[1]
- **"Cargo Cult"**: Template structure filled with content that doesn't encode intent. Fix: verify each section passes its quality test.[11]
- **"Missing the Architecture Layer"**: All constraints in the prompt, including ones that need code enforcement. Fix: every constraint answers "what enforces this?"[8][7]

SKILL.md Format Compliance

The generated file follows Claude Code's official SKILL.md format: YAML frontmatter with `name` and `description` fields, followed by markdown instruction content. The description is written to trigger on relevant queries (creating agent specs, converting prompts, reviewing specs). It references a `template.md` companion file for the blank form and an `examples/` directory for additional domain examples you can add over time.[12]

Recommended File Structure

```
~/.claude/skills/intent-engineering/
├── SKILL.md          (main skill - 917 lines)
├── template.md       (blank intent spec form)
```

```
└── examples/
    └── (add domain-specific examples as needed)
```

Usage Modes

The skill activates in four modes depending on user intent:

- **Write**: Drafts a new intent spec using the 9-section template, asks clarifying questions about domain/users/failure modes, runs the validation checklist.
- **Review**: Audits an existing spec against the 30-item checklist, checks for all 10 common mistakes and 5 anti-patterns, provides section-by-section feedback.
- **Retrofit**: Assesses an existing prompt-based skill, recommends a conversion level, performs the conversion, validates the result.
- **Compare**: Frames trade-offs using the Objective as decision criteria, evaluates options against Outcomes and Health Metrics, recommends based on the autonomy/risk framework.

References

1. The Intent Engineering Framework for AI Agents - The Intent Engineering Framework shows how to structure objectives, outcomes, health metrics, constr...
2. Agent-Ready Spec — Definition & Guide - Glossary - Pathmode - A specification structured so that an AI coding agent can execute it without follow-up questions, co...
3. Intent Engineering — Definition & Guide - Glossary - Pathmode - Intent Engineering is the practice of defining structured, evidence-backed specifications that AI co...
4. Paweł Huryn's Post - LinkedIn - Everyone talks about the importance of “intent” when building AI agents ... Here's the Intent Engine...
5. Multi-agent workflows often fail. Here's how to engineer ones that ... - Schemas define structure whereas action schemas define intent. MCP enforces both. Learn more about h...
6. Intent Engineering , Its the Foundation - LinkedIn - The I-5 Framework operationalizes the entire philosophy of Intent Engineering into a structured meth...

7. 7 AI Agent Failure Modes and How To Fix Them | Galileo - Seven critical failure modes in AI agents · Specification and system design failures · Reasoning loo...
8. Why AI Agents Break: A Field Analysis of Production Failures - Arize AI - Agents fail differently from software. They hallucinate data and enter silent loops. See eight commo...
9. Paweł Huryn's Post - LinkedIn - Everyone talks about the importance of “expressing intent” when building AI agents. Few explain what...
10. The Risk of Destructive Capabilities in Agentic AI - Noma Security - Learn more about agentic AI security risks and how excessive autonomy can lead to destructive, catas...
11. Intent Engineering for AI Agents: The Framework Methodology ... - A good prompt helps an AI complete a single task. A good framework enables an AI to operate autonomo...
12. Extend Claude with skills - Claude Code Docs - Claude Code skills follow the Agent Skills open standard, which works across multiple AI tools. Clau...